

机器学习第二次作业

姓名 周瑞奇 电子工程系 2025210719

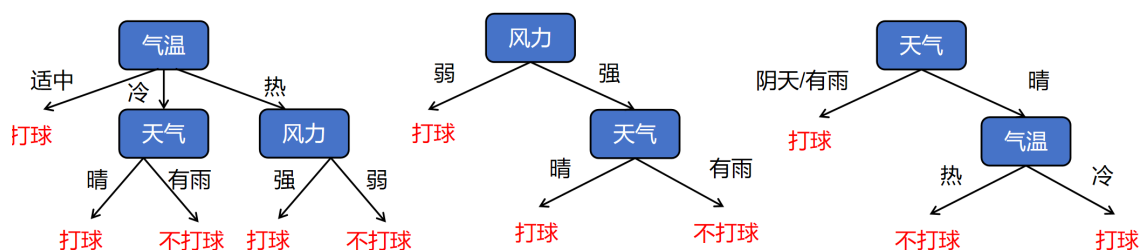
一、包装法 (Bagging)

1、决策树构建：

(1) 首先进行自助采样。不妨设采样结果为：

$$D_1 = D, D_2 = \{2, 3, 4, 5, 6, 6\}, D_3 = \{1, 3, 3, 4, 4, 6\}. \quad (1)$$

(2) 根据信息熵构建决策树（最大深度为2）分别如下：



2、天气晴朗，气温热，湿度高，风力弱时，三棵决策树的决策分别为不打球、打球、 不打球。根据众数投票的结果，最终结果为不打球。

3、Bagging 集成模型定义为：

$$f_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B f_b(x). \quad (2)$$

其中每个基学习器满足 $\text{Var}(f_b) = \sigma^2$ ，两个基学习器 (f_i, f_j) 的协方差为 $\text{Cov}(f_i, f_j) = \rho\sigma^2$ ，其中 ρ 为相关系数。故有

$$\begin{aligned} \text{Var}\left(\sum_{b=1}^B f_b\right) &= \sum_{b=1}^B \text{Var}(f_b) + 2 \sum_{1 \leq i < j \leq B} \text{Cov}(f_i, f_j) \\ &= B\sigma^2 + B(B-1)\rho\sigma^2. \end{aligned} \quad (3)$$

故集成模型的最终方差为：

$$\text{Var}(f_{\text{bag}}) = \frac{1}{B^2} \cdot [B\sigma^2 + B(B-1)\rho\sigma^2] = \frac{\sigma^2}{B} + \rho\sigma^2 \left(1 - \frac{1}{B}\right). \quad (4)$$

当 $\rho < 1$ 时，显然有 $\text{Var}(f_{\text{bag}}) < \sigma^2$ ，因此集成会减小方差。当 $\rho = 1$ 时， $\text{Var}(f_{\text{bag}}) = \sigma^2$ ，基学习器完全相关，集成不会降低方差。当 $\rho = 0$ 时， $\text{Var}(f_{\text{bag}}) = \frac{\sigma^2}{B}$ ，随着 B 增大，方差趋近于0，集成效果最好。

二、提升法 (Boosting)

1、第 t 轮时，手动执行AdaBoost分为以下几步：

- (1) 初始化权重为 $\frac{1}{N}$ ($t = 1$ 时)
- (2) 选出最优势分类器使得加权误差 ε_t 最小
- (3) 计算 α :

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right). \quad (5)$$

- (4) 更新权重 D :

$$D_{t+1}(n) = \frac{D_t(n)}{\sum_{i=1}^N D_t(n) \exp(-\alpha_t y_n h_t(x_n))} \exp(-\alpha_t y_n h_t(x_n)). \quad (6)$$

- (5) 输出最终分类器 (最后一轮迭代后):

$$H(x) = \text{sign} \left(\sum_{t=1}^3 \alpha_t h_t(x_n) \right). \quad (7)$$

重复上述步骤，可以写出每一轮的参数更新状态：

参数	p	θ	α
t=1	-1	3.5	0.4236
t=2	-1	9.5	0.4236
t=3	1	6.5	0.2027

最终的强分类器表达式为：

$$H(x) = \text{sign}(-0.4236 * \text{sign}(x - 3.5) - 0.4236 * \text{sign}(x - 9.5) + 0.2027 * \text{sign}(x - 6.5)). \quad (8)$$

预测结果为：

$$y_{pred} = \{1, 1, 1, -1, -1, -1, 1, 1, 1, -1\}. \quad (9)$$

预测准确率为100%。

我们手动编写了以下MATLAB代码来执行计算过程。

```
% Adaboost
clear
clc

%初始化设置
iter=3;
y=[1,1,1,-1,-1,-1,1,1,-1]';
x=[1,2,3,4,5,6,7,8,9,10]';
theta=x-0.5;
p=[-1,1]';
D=ones(10,1)*(1/10);
dec=0;
```

```

for i=1:1:iter
    [p_pred,theta_pred,loss_pred]=find_min(D,x,y,p,theta) %打印参数
    alpha=0.5*log((1-loss_pred)/loss_pred);
    h=p_pred.*sign(x-theta_pred);
    Z=0;
    for k=1:1:10
        Z=Z+D(i,1)*exp(-alpha*y(i,1)*h(i,1));
    end
    D=(D.*exp(-alpha.*y.*h))./(Z);
    dec=dec+alpha.*h;
end

H=sign(dec);
res=H-y %打印预测误差

function loss= error(D,x,y,p,theta)
loss=0;
for i=1:1:10
    h=p*sign(x(i,1)-theta);
    if h ~= y(i)
        loss=loss+D(i,1);
    end
end
end

function [p_final,theta_final,loss_temp]=find_min(D,x,y,p,theta)
p_final=p(1,1);
theta_final=theta(1,1);
loss_temp=error(D,x,y,p_final,theta_final);
for i=1:1:2
    for j=1:1:10
        loss=error(D,x,y,p(i,1),theta(j,1));
        if loss<loss_temp
            loss_temp=loss;
            p_final=p(i,1);
            theta_final=theta(j,1);
        end
    end
end
end
end

```

2、根据 $\alpha = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$ ，如果弱分类器的误差越小，则分配权重越大，反之则权重越小。由于AdaBoost每次选择当前权重下最优的弱分类器，且样本权重会聚焦于之前分类错误的样本，后续弱分类器针对性优化难分样本，因此组合后整体性能逐步提升。

3、AdaBoost在每轮计算时都对样本的权重进行调整。对前一步被正确分类的样本给予较小的权重，而对错误分类的样本则给予较大的权重，这样就迫使后续基学习器学习这些被错误分类的样本。最终通过加权组合多个弱分类器形成最终的强分类器，提升了模型的表达能力，有效降低了偏差。

三、聚类

1、分析如下：

- (a) 适合K-means，其数据分布表现为三个圆形团簇。
- (b) 适合DBSCAN，其数据的两叶密度相似，且间隔较远，可以用密度可达的方法区分。
- (c) 适合Spectral Clustering，可以通过图拉普拉斯矩阵的特征向量使得外圈和内圈的数据线性可分。
- (d) 适合Spectral Clustering，因为密度不均且数据团簇相隔较近，不适合使用K-means和DBSCAN。但Spectral Clustering能通过图的特征值分解将数据进行降维进而实现可分。

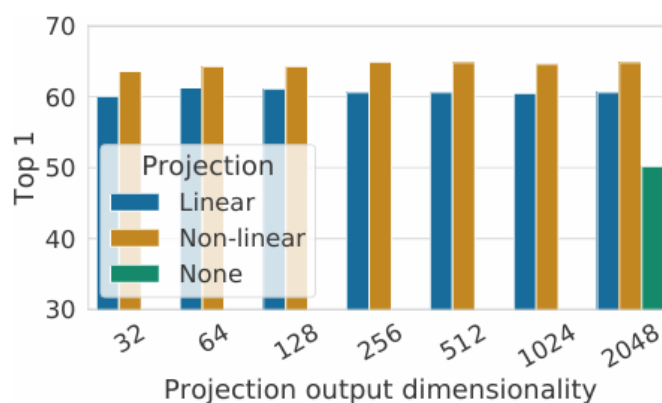
2、如下图所示：



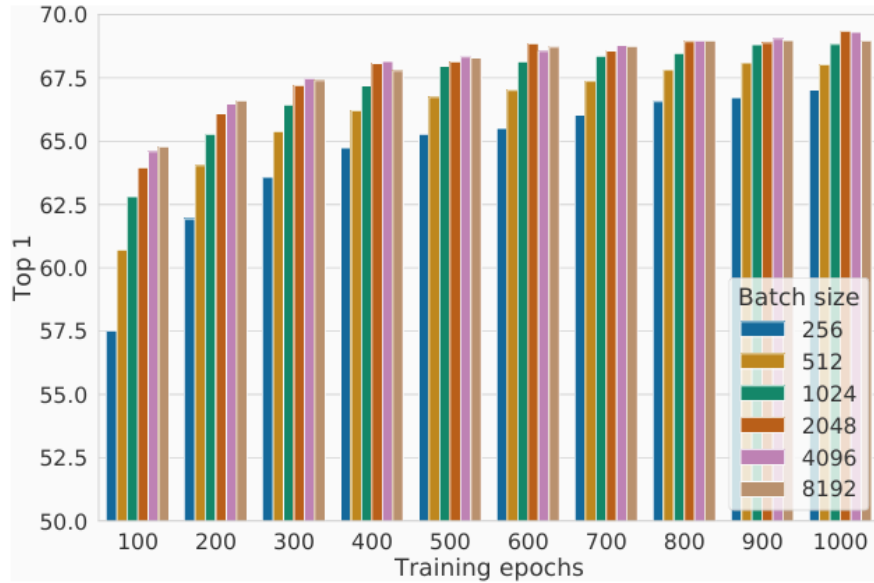
四、SimCLR

参考文献为 [2002.05709] A Simple Framework for Contrastive Learning of Visual Representations

1、投影头的作用是将编码器输出的特征映射到更适合对比学习的空间，增强特征的区分性。若移除投影头，则编码器特征可能未充分学习到实例间的判别性模式，导致特征表示的区分度、鲁棒性下降。相应的会造成下游任务上的性能降级。（论文Fig. 8）



2、batch 越大则负例数量越多、分布越丰富，对比监督信号更强，模型能学到更具区分度的特征。训练更长时间也能达到相似的效果。（论文Fig.9）



3、SimCLR的核心是让同一实例的不同图在特征空间中靠近，而不同的实例则尽量远离。在训练过程中，SimCLR采用了归一化温度缩放交叉熵作为损失函数：

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} 1_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)} \quad (10)$$

这样的操作会使得特征自然因为“拉近正例，推开负例”而形成不同的特征簇，进而实现了隐式聚类效果。

五、随机梯度下降

1、在SGD中，随机梯度的期望等于真实梯度，即：

$$E(g(x, \xi)) = \nabla_x F(x) \quad (11)$$

当 $x \rightarrow x^*$ 时， $\nabla_x F(x)$ 将会平稳地收敛到0，但随机梯度只是期望等于真实梯度，其本身引入了噪声，因此其本身会在0附近波动，而不是平稳收敛到0。

2、会收敛到一个与 η 有关的小邻域内，而不能准确地收敛到0。

3、需满足 $\eta_t \rightarrow 0$ 和 $(1 - \eta_t \mu)^t \rightarrow 0$ 。注意到

$$\begin{aligned} (1 - \eta_t \mu)^t \rightarrow 0 &\Leftrightarrow t \ln(1 - \eta_t \mu) \rightarrow -\infty \\ &\Rightarrow t \eta_t \mu \rightarrow +\infty \\ &\Rightarrow t \eta_t \rightarrow +\infty \end{aligned} \quad (12)$$

因此，需满足 $\eta_t \rightarrow 0$ 和 $t \eta_t \rightarrow +\infty$ 这两个条件。亦即： η_t 以比 $\frac{1}{t}$ 更慢的速度收敛到0。

六、Adagrad

1、对于稀疏特征， s_t 累积较小，会为其分配较大的学习率以加快更新，而对于稠密特征，梯度更新十分频繁， s_t 累积较大，会为其分配较小的学习率以减少震荡。而单一全局学习率的SGD对稀疏特征和稠密特征采用相同的更新幅度，为了保证收敛性（减少震荡）就不得不选择更小的学习率，这导致在稀疏特征上往往更新过慢。因此，Adagrad通常比单一全局学习率的SGD收敛更快。

2、学习率不断衰减会使得更新幅度越来越小，最终完全收敛到梯度为0的局部最优点。但这样容易导致解陷入较差的局部最优点而无法“挣脱”出来，导致优化效果不佳。相比之下，带有动量的 SGD 引入动量项累积历史梯度方向，能够有效地逃离较差的局部最优点。

七、近端梯度下降

可以结合。结合之后算法在第 k 步的更新步骤为：

$$x^k = \text{prox}_{t_k h}(x^{k-1} - t_k g(x^{k-1})). \quad (13)$$

其中 $g(x^{k-1})$ 是 $f(x)$ 在 x^{k-1} 处的随机梯度估计。